

Project Brief — Sales Enablement Pro

Capstone pitch · Devin Bajaj · drafted 2026-07-14

1. Business Problem

Project title

Sales Enablement Pro — an AI deal room that turns a rep's own uploaded sales artifacts into a cited, actionable next-steps plan.

Target user

Individual B2B sales representatives and account executives working 5–20 active deals at once, at a company without a dedicated sales-enablement team or a well-maintained CRM.

Domain or industry

B2B software sales / sales enablement.

The business problem

A rep's most valuable context about a deal is scattered and unstructured. It lives in meeting notes typed during a call, a prospect's RFP PDF, a company one-pager, a pricing sheet, and a discovery-call summary. After every customer interaction the rep has to re-read all of it and answer one question: **“what do I actually do next to move this deal forward?”**

That synthesis is manual, slow, and inconsistent. It happens late at night or not at all. The result is the single most common way deals die: not a lost bake-off, but a deal that quietly stalls because nobody sent the follow-up, answered the security questionnaire, or looped in the economic buyer.

Sales Enablement Pro fixes the synthesis step. A rep creates a deal, uploads the artifacts they already have, and the app generates a **source-backed action plan**: a short situation summary, a sequenced list of next steps, and a checklist of action items — every claim traceable to the exact passage in the rep's own documents that justifies it.

Why the problem matters

Stalled deals are expensive and invisible. A rep who misses a follow-up commitment does not get an error message; they get a quiet loss two months later. Reducing “time from customer conversation to a concrete written next step” from *hours or never* to *under a minute* directly protects pipeline. It also compounds: the plan is persistent, so the rep's next prep session starts from the last plan rather than from a blank page.

What users currently struggle with today

- Context is spread across a notes app, an email attachment, a PDF, and their memory.
- Re-reading a 30-page RFP to extract “what did they actually ask us to commit to” is a tax paid on every deal.

- Generic AI chatbots will happily *invent* a plausible next step, and a rep cannot safely act on an AI claim they cannot trace back to something the customer actually said.
- Action items live in a rep's head or a scratch text file, disconnected from the document that motivated them.

What success looks like

- A rep uploads a meeting note and an RFP to a deal and gets a usable action plan in under 60 seconds.
- Every next step in the plan shows the source document and the quoted passage behind it, so the rep can verify before acting.
- The rep works the plan: checking items off, editing them, adding their own.
- When the rep has not uploaded enough context, the app **says so** rather than fabricating a plan.
- A rep can never see another rep's deals, documents, or plans.

Why this is a good fit for a full-stack AI application

The problem is fundamentally *retrieval-shaped*. The answer to “what do I do next” is not general world knowledge — it is latent in a specific, private, small set of documents that only this user has. That is exactly what RAG is for, and it is exactly what a raw LLM cannot do. It also demands real authentication (deal documents are confidential), real relational structure (deals own documents own chunks; plans own items and citations), and real CRUD (an action plan you cannot edit is a report, not a tool). The AI feature is not decoration — remove it and the product is just a folder of PDFs.

2. Problem-Solving Process

MVP feature list

1. Email/password signup, login, logout, session-check route.
2. Deal CRUD — create, view, update, delete deals. (*This is the protected user-owned resource.*)
3. Document upload to a deal (PDF / TXT / MD), typed as `meeting_note`, `company_info`, or `rfp`. View and delete documents.
4. Automatic ingestion pipeline on upload: extract text → chunk → embed → index.
5. **Generate Action Plan** — the core AI/RAG feature. Retrieves from *only this deal's* documents and produces a situation summary, sequenced next steps, and discrete action items.
6. Every next step and action item displays its supporting sources: document name plus the quoted chunk that justifies it.
7. Action items are persistent and editable — check off, edit, reprioritize, delete, add your own.
8. Explicit weak-context handling: if the deal has no documents, or retrieval finds nothing above a relevance floor, the app refuses to generate and tells the user what to upload.
9. Authorization enforced on every route: a user touching another user's deal gets a 404.
10. Deployed to a public URL with seed data and a demo account.

Stretch feature list

(Explicitly out of MVP. Attempted only if the deployed MVP is stable.)

- **Presenton integration** — generate a customer-facing deck from the deal’s documents and action plan. Highest-value stretch, highest risk (see §4).
- Free-form Q&A against a deal’s documents (“what did they say about security?”), reusing the same retrieval layer.
- Cross-deal retrieval — pull in a shared company-wide battlecard corpus alongside the rep’s private docs.
- DOCX support and OCR for scanned PDFs.
- Regenerate a plan while preserving already-completed action items.
- Export the action plan to Markdown / email.

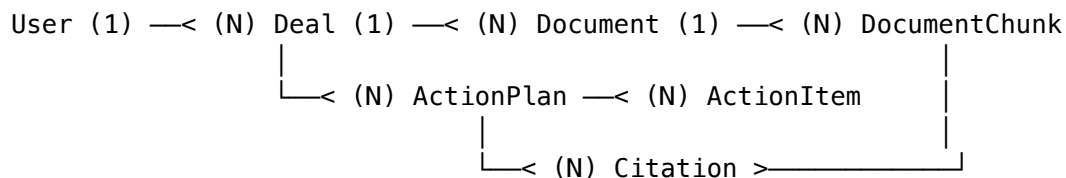
Authentication approach

Flask session cookies via **Flask-Login**, with passwords hashed using **bcrypt**. Frontend and backend are separate Render services, so cookies are configured SameSite=None; Secure; HttpOnly with CORS locked to the frontend origin and supports_credentials=True.

I chose sessions over JWT deliberately: the rubric wants a “current user / session check” route, which is `GET /api/me` and falls straight out of Flask-Login. Hand-rolled JWT in a two-week window is a good way to ship a subtle auth bug, and `HttpOnly` cookies are not readable by XSS the way a token in `localStorage` is.

Authorization is the part I will not hand-wave. Every query for a user-owned record filters by `user_id` *in the query itself* (`Deal.query.filter_by(id=deal_id, user_id=current_user.id)`), never by fetching-then-checking. A missing record and a forbidden record both return **404**, so the API does not leak the existence of other users' deals. Retrieval carries the same rule: the vector search is filtered by `user_id` *and* `deal_id` in its metadata filter, and the chunks that come back are re-validated against SQL ownership before they ever reach the prompt. One bug in the vector filter should not be enough to leak another rep's documents into an LLM call.

SQL data model and relationships



Model	Key fields	Relationships
User	id, email (unique), password_hash, name, created_at	has many Deals
Deal	id, user_id (FK), name, company, stage, value, close_date, created_at	belongs to User; has many Documents, ActionPlans

Model	Key fields	Relationships
Document	id, deal_id (FK), user_id (FK, denormalized for query-time authz), filename, doc_type, raw_text, uploaded_at	belongs to Deal; has many DocumentChunks
DocumentChunk	id, document_id (FK), chunk_index, content, embedding (JSON float array)	belongs to Document; referenced by Citations
ActionPlan	id, deal_id (FK), user_id (FK), summary, next_steps (JSON), model_used, generated_at	belongs to Deal; has many ActionItems, Citations
ActionItem	id, action_plan_id (FK), title, detail, priority, due_date, status, is_user_created	belongs to ActionPlan
Citation	id, action_plan_id (FK), chunk_id (FK), quote, relevance_score	joins ActionPlan ↔ DocumentChunk

Six app-specific models against a requirement of two, but none are filler: Document/DocumentChunk is the knowledge base, ActionPlan/ActionItem is the persistent AI artifact, and Citation is the many-to-many join that makes source-backed display possible. Citation is the model that earns the “meaningful relationship” line — it is what lets a generated action item point back at the exact passage in the exact document that produced it.

CRUD target: Deal gets the full four verbs, and ActionItem gets create/update/delete so the rep can actually work the plan.

Frontend views

Route	Purpose	Protected
/login, /signup	Auth forms	No
/deals	Dashboard — deal list, create deal, stage/value at a glance	Yes
/deals/:id	Deal detail — document upload + list, plan history, “Generate Action Plan”	Yes
/plans/:id	The payoff view — summary, next steps, action-item checklist, citation cards	Yes

Handled explicitly: loading state during generation (a 5–15s LLM call needs a real spinner and a disabled button, not a frozen page), the empty state (a deal with no documents, which points at the

uploader), the weak-context state (documents exist but do not support a plan), and the error state (upload failed, model call failed).

Backend API routes

Auth

- POST /api/signup · POST /api/login · DELETE /api/logout · GET /api/me

Deals (*all protected, all scoped to current_user*)

- GET /api/deals · POST /api/deals · GET /api/deals/:id · PATCH /api/deals/:id · DELETE /api/deals/:id

Documents

- POST /api/deals/:id/documents (multipart upload; triggers ingestion) · GET /api/deals/:id/documents · DELETE /api/documents/:id

AI / RAG

- POST /api/deals/:id/action-plans — the RAG call. Retrieves, prompts, generates, persists, returns the plan with citations.
- GET /api/deals/:id/action-plans · GET /api/action-plans/:id · DELETE /api/action-plans/:id

Action items

- POST /api/action-plans/:id/items · PATCH /api/action-items/:id · DELETE /api/action-items/:id

AI/RAG workflow

Ingestion (on upload)

1. Extract text — pypdf for PDFs, direct read for TXT/MD. If extraction yields near-zero text (a scanned PDF), reject the upload with a clear message rather than silently indexing an empty document.
2. Chunk with LangChain's RecursiveCharacterTextSplitter (~800 chars, ~100 overlap).
3. Embed each chunk with Google's embedding model via langchain-google-genai.
4. **Write chunks + embeddings to Postgres** (source of truth) **and** add them to the Chroma collection with metadata {user_id, deal_id, document_id, chunk_id}.

Retrieval + generation (on “Generate Action Plan”)

1. Build a retrieval query from the deal's context — the deal name, company, stage, plus a fixed task framing (“what are the next steps to advance this deal”).
2. Similarity search in Chroma, **filtered to user_id AND deal_id**, top $k \approx 8$.
3. **Weak-context gate**. If zero chunks return, or the best similarity score falls below a tuned floor, stop here. Return a structured insufficient_context response naming what is missing. No model call, no invented plan.
4. Re-validate every retrieved chunk_id against SQL ownership. Defense in depth.

5. Construct the prompt: system instructions (“you are a sales-enablement assistant; ground every claim in the numbered sources; if the sources do not support a step, do not invent it”), the deal metadata, and the retrieved chunks as numbered source blocks [1]...[8].
6. Call Gemini through LangChain with **structured output** — a Pydantic schema for {summary, next_steps[], action_items[{title, detail, priority, source_ids[]}] } — so the model returns parseable JSON rather than prose I have to regex.
7. Persist the ActionPlan, its ActionItems, and a Citation row per source_id the model cited, linking back to the real DocumentChunk.
8. Return the plan with citations hydrated to {document filename, doc_type, quoted chunk text, score}.

Source display. Each next step and action item renders with numbered citation chips. Clicking one expands the actual quoted passage and names the document it came from. The rep can see the sentence in their own meeting note that produced the recommendation.

Knowledge base / source content

The knowledge base is **the user’s own uploaded deal artifacts** — meeting notes, company/prospect info, and RFPs — scoped per deal. This is what makes the app defensible: the retrieval corpus is private, per-user, and per-deal, which forces the authorization story into the AI pipeline itself rather than leaving it at the API boundary.

For seeding, demos, and grading I will write a realistic synthetic deal: a discovery-call meeting note, a prospect company profile, and a 5–8 page RFP with real questions and deadlines buried in it. That gives a reviewer something to generate against immediately, and gives me a stable corpus to evaluate answer quality against while building.

Persistent AI-related feature

The **ActionPlan** is the persistent artifact. It is not a chat message that scrolls away — it is a stored, revisitable record with its own URL, its own citations, and a checklist the user works over days. Plan history per deal is retained, so a rep can see how the deal’s next steps evolved after each new meeting note. The ActionItem rows are user-editable, which means AI output becomes user-owned data the moment it lands.

Expected user flow

Rep signs up / logs in

- lands on protected Deals dashboard
- creates a deal ("Acme Corp – Platform Renewal")
- uploads a discovery-call meeting note (PDF) and Acme's RFP
- ingestion runs: extract → chunk → embed → index (Postgres + Chroma)
- clicks "Generate Action Plan"
- Flask retrieves top-k chunks, filtered to this user + this deal
- weak-context gate passes; prompt is built from the retrieved chunks
- Gemini returns structured JSON: summary, next steps, action items, source ids
- plan + citations persist to Postgres
- React renders the plan; each step shows the document and quoted passage behind it
- rep expands a citation, confirms the claim is real, edits one item, adds another

→ rep checks items off over the following week; plan history grows with the deal

Deployment platform

Render, both services. Flask API as a web service, React as a static site, one Render Postgres instance. Gemini via the Google AI Studio free tier. Chroma runs in-process inside the Flask service.

Why these tools fit

- **Flask + SQLAlchemy + Flask-Migrate** — the relational model is the point of this app, and migrations keep the schema honest as it changes mid-build.
- **Postgres over SQLite** — Render's filesystem is ephemeral; a SQLite file would be deleted on every deploy. This is not a preference, it is a hosting constraint.
- **Chroma + LangChain** — Chroma gives me metadata-filtered similarity search out of the box, which is precisely the primitive the per-deal authorization model needs. LangChain's splitter, Gemini bindings, and structured-output helpers remove three pieces of plumbing I would otherwise hand-write under deadline.
- **Gemini via langchain-google-genai** — a free tier covering *both* generation and embeddings, which is the whole RAG pipeline on one key. Structured output means I get typed JSON instead of parsing prose.
- **Flask-Login sessions** — the session-check route is a rubric requirement and a one-liner here; HttpOnly cookies dodge the localStorage-token XSS problem.

3. Timeline and Scope

Two weeks. **Pitch 2026-07-16 · critique at ~90% on 2026-07-24 · final showcase 2026-07-28.** The governing rule: **deployed by day 5, not day 12.** Deployment is where capstones die, and I would rather deploy a skeleton early and redeploy 20 times than discover Chroma's persistence problem on the last night.

Phase 1 — Plan and pitch · Jul 14–16

Problem definition, MVP scope, data model, route plan, frontend plan, RAG plan, deployment plan, knowledge-base decision, risk list. **All of the above is this document.** Also in this phase: write the synthetic seed corpus (meeting note, company profile, RFP), and spike the two riskiest unknowns — a Gemini structured-output call and a Chroma metadata-filtered query — as throwaway scripts before committing to the architecture.

Phase 2 — Build MVP · Jul 17–23

- **Jul 17** — Flask app factory, SQLAlchemy models, migrations, seed script. Auth end to end (signup, login, logout, /api/me, bcrypt, Flask-Login).
- **Jul 18** — Deal CRUD with ownership filtering. React auth context, protected routes, login/signup, deals dashboard. **Deploy both services to Render today**, with auth working, even though nothing else does.
- **Jul 19** — Upload + ingestion: text extraction, chunking, embedding, dual write to Postgres and Chroma. Document list and delete.

- **Jul 20–21** — The RAG endpoint. Retrieval with metadata filtering, weak-context gate, prompt construction, structured Gemini call, persistence of plan/items/citations.
- **Jul 22** — Plan view in React: summary, next steps, citation chips with expandable source passages. Action-item CRUD and check-off.
- **Jul 23** — Loading/empty/error states, seed the demo account, redeploy, verify the full flow on the public URL. **This is the 90% build for critique.**

Phase 3 — Finish and showcase · Jul 24–28

- **Jul 24** — Post-critique triage. UX refinement. Deliberate authorization review: log in as user B and attempt to read, edit, and delete every one of user A's deals, documents, plans, and items — by direct ID — and confirm each returns 404.
- **Jul 25** — AI response quality pass: tune k , the relevance floor, and the prompt against the seed corpus. Verify citations actually point at passages that justify their claims.
- **Jul 26** — README (all fifteen required sections), `.env.example`, setup and run instructions, deployment notes with honest production limitations.
- **Jul 27** — Final deploy verification, demo video, written reflection.
- **Jul 28** — Submit. Stretch features only if everything above is genuinely done.

Technical risks and blockers

1. **Chroma persistence on ephemeral hosting.** Render's free filesystem does not survive a restart, so a Chroma directory written to local disk is *gone* on every redeploy. Because the corpus is now user-uploaded rather than a fixed seeded set, I cannot just rebuild it from files in the repo. **Mitigation:** Postgres is the source of truth. DocumentChunk stores the chunk text *and* its embedding vector. Chroma is treated as a rebuildable index, not a database — it is populated at app startup from Postgres and written to on each upload. Losing the Chroma directory costs a few seconds of boot time and nothing else. This is the single most important design decision in the project and it is why DocumentChunk.embedding exists.
2. **Gemini free-tier rate limits and structured-output reliability.** Free-tier requests-per-minute caps are real, and a demo that 429s in front of a grader is a bad day. **Mitigation:** embed in batches, cache embeddings in Postgres so a document is never re-embedded, keep k small, and wrap the generation call in retry-with-backoff. If structured output proves flaky, the fallback is a JSON-mode prompt plus a validating parser — but I will find that out during the Phase 1 spike, not on Jul 26.
3. **PDF text extraction quality.** A scanned or image-only PDF yields no text and would silently produce an empty knowledge base. **Mitigation:** MVP supports text-based PDFs only; extraction that returns near-zero characters rejects the upload with an explanatory error. OCR is a stretch goal.
4. **Presenton integration (stretch).** I have not yet verified what Presenton requires to run — whether there is a hosted API I can call with a key, or whether it must be self-hosted as a separate Docker service. If it is the latter, hosting it free alongside two Render services is likely infeasible. **Mitigation:** timeboxed to Jul 28, cut without hesitation, and I will verify its actual deployment requirements before writing a single line against it.
5. **Render free tier's rough edges.** Web services sleep after inactivity, so a grader's first request may take ~50 seconds. Free Postgres instances also expire after roughly a month. **Mitigation:** document both in the README under production limitations, and confirm the database's expiry date lands well past the grading window.

Concepts to review before building

Flask-Login session config for cross-origin cookies (SameSite=None, Secure, CORS credentials); Chroma's metadata where filter syntax; LangChain's structured-output binding for Gemini; Flask-Migrate against Postgres on Render; multipart file upload handling in Flask.

Where I expect to revise after the critique

Most likely the AI output itself — whether the generated next steps are genuinely *useful* to a rep or merely fluent, and whether the citation display actually builds trust or just adds noise. Prompt and UI, in that order. I also expect feedback on whether plan history per deal is valuable or clutter.

What I will cut if the project runs long

In order: (1) Presenton, first and without hesitation. (2) Free-form Q&A. (3) Plan history — keep only the latest plan per deal. (4) Deal PATCH — CRUD on `ActionItem` alone satisfies the rubric. (5) Multiple document types — meeting notes only. I will not cut, under any circumstances: the weak-context gate, the citation display, or the authorization filtering. Those three are the difference between this project and a chatbot glued to a CRUD app.

4. Technical Feasibility and Risk Plan

What must work for the MVP to be successful

A reviewer, on the public URL, can: register → log in → create a deal → upload a document → generate an action plan → see the answer with its supporting sources → edit and check off action items → log out. And a second account cannot see any of the first account's data. That sentence is the definition of done. Everything else is negotiable.

Intentionally out of scope

Team/org accounts and sharing. Real CRM integration (Salesforce, HubSpot). Email or calendar sync. Multi-user collaboration on a deal. Audio/call-recording transcription. OCR. Deck generation, unless the stretch lands.

The most technically risky parts

Ranked honestly: (1) Chroma persistence across ephemeral deploys — mitigated by the Postgres-as-source-of-truth design above, and the reason that design exists. (2) The deployed cookie/CORS handshake between two different Render origins, which is the classic “works locally, 401s in production” trap. (3) Gemini free-tier limits under demo load. (4) Presenton, which is why it is a stretch goal and not a feature.

How I reduce risk early

Before writing application code, I spike the two things I have never done: a Gemini structured-output call and a Chroma metadata-filtered query. And I deploy both services to Render on **day 5**

with auth working and nothing else — because the cross-origin cookie problem is the failure I most expect, and I want to hit it on Jul 18 with ten days of runway, not on Jul 27 with none.

How I keep the AI feature from becoming a disconnected chatbot

By construction. The AI does not answer questions about the world; it reads *this user's documents for this specific deal* and produces *the artifact the app exists to produce*. Its output is not a chat bubble — it is an `ActionPlan` row with `ActionItem` children that the user then edits, checks off, and returns to. There is no chat page. Retrieval is hard-scoped to a deal, so the feature literally cannot function detached from the deal it belongs to. Delete the AI feature and the app is a document folder; that is the test, and it passes.

How I check the AI response is useful and source-backed

The seed corpus is the evaluation set. I plant specific, checkable facts in it — a security-questionnaire deadline in the RFP, a promised follow-up in the meeting note, a named economic buyer in the company profile — and I verify the generated plan surfaces them and cites the right chunk. Concretely, I check three things every time I touch the prompt: does every action item carry at least one citation; does the cited passage actually justify the claim; and does a deal with an irrelevant document correctly trip the weak-context gate instead of confabulating. The gate is tested by generating against a deal whose only upload is unrelated content — the correct output is a refusal.

How I deploy frontend and backend

Two Render services from one GitHub repo. Flask API as a web service (gunicorn, Python 3.11, `pip install -r requirements.txt`, migrations run on release). React as a static site (`npm run build`, publish `dist/`, SPA rewrite to `index.html`). Postgres as a managed Render instance.

How I configure production environment variables

All secrets are Render dashboard environment variables — never committed. A `.env.example` in the repo documents every key with placeholder values. Backend needs `SECRET_KEY`, `DATABASE_URL`, `GOOGLE_API_KEY`, `FRONTEND_ORIGIN`, `FLASK_ENV`. Frontend needs `VITE_API_URL`. `.env` is in `.gitignore` from the first commit, and I will verify no key was ever committed before submitting — checking the history, not just the working tree, because a secret deleted in a later commit is still a leaked secret.

How the deployed app reaches its dependencies

- **Database** — Render Postgres via `DATABASE_URL` (normalizing the legacy `postgres://` prefix SQLAlchemy rejects).
- **Vector store** — Chroma, in-process in the Flask container, rebuilt at startup from `DocumentChunk` rows in Postgres. No separate service, no separate credentials, nothing to fall out of sync — because Postgres is authoritative and Chroma is derived.
- **Model service** — Google AI Studio over HTTPS with `GOOGLE_API_KEY`, called from the back-end only. The key never reaches the browser.

How I confirm the deployed app supports the full user flow

A written smoke-test checklist run against the **public URL** — not localhost — after every significant deploy, walking the definition-of-done flow above end to end. Plus the authorization check performed as a *deliberate attack*: logged in as user B, hit user A's deal, document, plan, and action-item IDs directly, across every verb, and confirm 404 on each. I will run that checklist one final time on Jul 27 and record the demo video against the deployed app, so the video is itself evidence the public deployment works.